

Develop Small, Deploy Large: A Coroutine-Bridge Harness that Transfers Reliability Across Model Scale on CAR-bench

Ivan Matveev

Proxima Ultra

i.a.matveev@gmail.com

Abstract

CAR-bench measures whether tool-using agents stay reliable under real-world uncertainty. We submit a coroutine-bridge harness whose only model action is to emit a Python program that blocks and resumes in place across evaluator tool exchanges, so deterministic policy lives as executable logic in the tool layer rather than as prompt rules. Our Track 1 methodology treats model scale like a train/test split: the harness is engineered and regression-tested against a small, fast model (gpt-oss-120b) and then deployed unchanged on a larger model (OpenAI gpt-5.5). Because the harness encodes transferable task structure, the identical agent runs on the larger model with no architecture change and resolves the residual cases the small model handled unreliably. The design’s efficiency is model-independent: two model calls carry a full multi-turn task on both models. Since the larger model’s calls are an order of magnitude more expensive in latency, the bridge’s call-minimization matters more the larger the model. The single static prompt also stays byte-identical across calls and across tasks, so most input tokens are served from provider prefix caches — a property shared by Cerebras (73% cached across our development runs) and the OpenAI route this track uses.

1 Introduction

CAR-bench evaluates the *consistency* and *limit-awareness* of LLM agents across 58 interconnected tools, 19 domain policies, and 254 public tasks in base, hallucination, and disambiguation categories [Kirmayr *et al.*, 2026]. The Open track ranks agents by hidden-set Pass³, which credits a task only when all three trials pass, and additionally rewards innovation in harnessing and reliability design.

Our submission is built on one idea applied twice. The model’s only action is to emit an executable Python program [Wang *et al.*, 2024]; a coroutine bridge suspends that program when it calls a tool, emits the official Agent-to-Agent (A2A) tool call, and resumes the same stack frame when results return — with no new model call. Because the

action surface is code, deterministic CAR-bench policies are compiled into the tool layer as ordinary logic rather than written as prompt rules the model must remember.

The Track 1 contribution is a *methodology*, stated as an analogy to a train/test split over model scale. We “train” the harness — hand-engineer it and lock it with regression tests — against a small, cheap, fast model where iteration is inexpensive, and “test” it by deploying the identical agent on a larger model. A change is accepted only if it improves the structure both models can use, never if it patches a weakness specific to the small one. The result is an agent whose reliability transfers upward across a large capability gap without re-engineering.

2 The Coroutine Bridge

CAR-bench externalizes tool execution to the evaluator so it can score the tool trajectory and inject environment perturbations. Each batch of tool calls the agent emits returns as a fresh A2A message. Batching keeps independent calls cheap, but a chain of dependent calls costs a conventional next-action agent one model call per round of results. The bridge removes this tax: the model emits one program, and tool results are delivered back into the blocked Python frame, which resumes without a model invocation (Figure 1). One program drives many sequential and parallel tool round-trips per model call. Independent reads are issued together through `batch([...])`, which collapses independent round-trips into one parallel A2A message. Every tool call still appears in the official trajectory, so the technique is fully compliant. The model emits its action as a plain-text JSON object parsed by the runtime, rather than through grammar-constrained or native structured decoding, following evidence that constrained decoding trades a small accuracy cost for token savings [Matveev, 2026; Schall and de Melo, 2025]; malformed actions are caught by a bounded schema-repair retry.

Policy as code. Many CAR-bench policies are deterministic given grounded facts: whether opening a window past 25% needs an air-conditioning confirmation, whether high beams are blocked while fog lights are on, which route default applies to a new segment. The code-action surface lets us encode these as logic in the tool layer — guarded wrappers that fire whether the model calls the public name

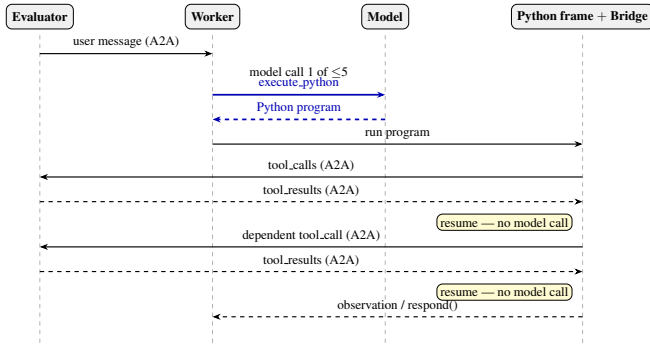


Figure 1: One user turn. Blue = model call; black solid = request, dashed = return. A single `execute.python` program drives several evaluator tool round-trips; each returning tool result resumes the blocked Python frame with no new model call. The loop is bounded at five sequential model calls (Track budget).

or the helper, a confirmation state machine that stores the fully grounded pending action, live tool-surface membership checks for missing capabilities, and result normalization that gives stable field names. The runtime owns facts and strict policy; the model owns interpretation of the request. A policy in the prompt can be forgotten under attention pressure; a policy in tool code is enforced by construction, at zero model-attention cost. This is what makes the harness transfer across models: the enforced structure is identical regardless of which model reasons over it.

3 Develop Small, Deploy Large

We treat model capability as a train/test axis. The small model (`gpt-oss-120b`) is the development crucible: cheap and fast enough to run full three-trial public-test sweeps repeatedly, so the harness can be hardened against real failures. Two rules govern what ships.

First, a change is accepted only if a *stronger* model already outperformed a *weaker* one on the bare prompt, before any domain skill. The bare-prompt ordering (stronger > weaker) is the signal that a task rewards reasoning and structure rather than memorized scaffolding. A change that improved the small model while regressing the large one is rejected as overfitting to the small model’s weaknesses.

Second, when the small model failed a task, we reproduced it on `gpt-5.5` before acting. If the stronger model passed, the ceiling was reachable within the current architecture and the fix was to encode the missing determinism as tool code (policy as code). If both failed, the defect was in existing harness and was repaired there rather than patched over. Candidate changes were judged on consistent-failure sets across runs, never single noisy trials, because near the plateau one trial moves several points on evaluator/simulator variance alone.

For Track 1 we then *deploy* the identical agent — same image, same domain skill, no architecture change — on `gpt-5.5`, selected purely through environment variables. Nothing about the harness is model-specific, so the larger model inherits the same enforced policy and normalization and applies stronger interpretation on top.

Per task (median)	<code>gpt-oss-120b</code>	<code>gpt-5.5</code>
Model calls	2	2
A2A turns	7	10
Model latency / call	~1 s	~10 s
Input tokens	78k	76k
Output tokens	1.4k	0.9k
Thinking tokens	—	0.15k

Table 1: The call-decoupling is model-independent: two model calls carry a full multi-turn task on both models (7 vs. 10 A2A turns). `gpt-5.5` figures are measured over the OpenAI API in our runs; all three token fields, including `thinking.tokens`, populate.

4 Results

The development benchmark is the full public test split on the small model (`gpt-oss-120b`, 125 tasks, 3 trials): **Pass¹ 93.3%** and **Pass³ 87.3%** (per-split Pass¹: base 90.0, hallucination 98.7, disambiguation 90.7). We do not self-evaluate the hidden set; the numbers below characterize the deployed `gpt-5.5` agent from our own runs. The user simulator and policy judge are Gemini 2.5 Flash, the organizers’ default.

The efficiency is structural. Table 1 shows the coroutine bridge holds a median of two model calls per task on *both* models, even though the tasks require seven to ten A2A tool round-trips. A conventional next-action agent would issue one model call per round-trip. The saving is a structural property of the harness that holds on any model.

Call-minimization pays off more at scale. The larger model’s completions are roughly an order of magnitude slower (~10 s vs. ~1 s per call in our measurements over the OpenAI API). Every model call the bridge avoids therefore saves about ten seconds on `gpt-5.5`. A per-turn baseline on the same tasks would issue roughly ten model calls instead of two; the bridge removes on the order of eighty seconds of sequential model latency per task. The more expensive the model, the more the structure is worth — the same design that trims latency on a fast small model trims far more on a slow large one.

Reliability transfer. During development, `gpt-5.5` served as the stronger-model oracle on residual failure clusters — tasks curated precisely because the small model was unreliable on them. On the final agent it resolves those targeted cases and passes the digest-pinned GHCR validation smoke. The Open-track Pass³ is what the organizers measure on the hidden set; our evidence is that the harness developed on the small model deploys unchanged and improves on the larger one.

Prefix-cache efficiency. The same one-big-prompt design that transfers policy across models also makes the prompt cache-friendly on any provider. The system prompt is one static block (rules, domain skill, full tool catalog); per-task state is appended only at the tail, so the cacheable prefix is byte-identical across calls and, crucially, *across tasks* — we never swap skills, which would break it. Once warm, per-minute cache rates reach **88.7% (p90) and 99.9% at peak**; across all development runs on Cerebras (918M input tokens)

the token-weighted rate is 73.2%, held down by prompt edits during development. OpenAI’s route applies the same automatic prefix caching, so the `gpt-5.5` deployment inherits the same class of saving; combined with two model calls per task, real input compute is a small fraction of the nominal per-task tokens.

Compute compliance. The internal loop is capped at five sequential model calls per step; observed usage is a median of two (p90 four). Mean input plus output stays well under the budget. Token and latency accounting is reported through `turn_metrics` on each concluding response, aggregated across intermediate tool exchanges, so per-task totals are complete; `prompt_tokens`, `completion_tokens`, and `thinking_tokens` all populate on the `gpt-5.5` route.

5 Limitations and Conclusion

We report a full public-test evaluation on the development model and a targeted plus smoke validation on the deployment model; a full self-evaluation on `gpt-5.5` was not run because the hidden set is organizer-scored and a full sweep on a frontier model is costly. Worker state is in memory and not recoverable across process restarts. Latency figures for `gpt-5.5` are wall-clock over the public OpenAI API and include network and provider queueing.

The submission makes one architectural bet and reuses it across a large capability gap: put deterministic policy in code the model calls, minimize the number of expensive model calls, and let a small fast model do the development that a large model then benefits from unchanged. The same image serves both the Cerebras Track 2 and this Open-track submission, differing only by environment variables — direct evidence that the reliability lives in the harness and transfers across models.

References

- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, United States, July 2026. Association for Computational Linguistics.
- [Matveev, 2026] Ivan Matveev. Token-oriented object notation vs JSON: A benchmark of plain and constrained decoding generation, 2026.
- [Schall and de Melo, 2025] M. Schall and G. de Melo. The hidden cost of structure: How constrained decoding affects language model performance. In *Proceedings of Recent Advances in Natural Language Processing (RANLP)*, pages 1074–1084, Varna, Bulgaria, 2025.
- [Wang *et al.*, 2024] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.